



Cyberscope

A *TAC Security* Company

Source Code Review Report

Crush Wallet

July 2026

Audited by © cyberscope

Table of Contents

Table of Contents	1
Risk Classification	2
Review	3
Audit Updates	3
Overview	4
Source Code Review Scope	4
Findings Breakdown	6
Diagnostics	7
NDSWU - No Docker Secrets Were Used	8
Description	8
Recommendation	9
NAI - No Authentication Implemented	10
Description	10
Recommendation	11
BINPBD - Base Image is Not Pinned By Digest	12
Description	12
Recommendation	13
VOC - Vulnerable and Outdated Component	14
Description	14
Recommendation	15
SE - Swagger Enabled	17
Description	17
Recommendation	18
ACAOHWW - Access-Control-Allow-Origin Header with Wildcard.	19
Description	19
Recommendation	20
IME - Insecure Methods Enabled	21
Description	21
Recommendation	22
Summary	23
Disclaimer	24
About Cyberscope	25

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Source	Source Code Files
---------------	-------------------

Audit Updates

Initial Audit	22nd February 2026 - 16th March 2026
Revalidated Audit	09 th June 2026 – 09 th June 2026
Revalidated Audit	07 th July 2026 – 08 th July 2026

Overview

Cyberscope has conducted a comprehensive Source Code Review for “Crush Wallet”. The source code review was performed using a manual, risk-based approach, supplemented by automated analysis where appropriate. The assessment focused on security-critical components and high-risk code paths, including authentication mechanisms, sensitive data handling, external integrations, and business logic. The review began with an architectural understanding of the application, followed by systematic examination of source files to identify potential vulnerabilities, insecure API usage, hardcoded secrets, and improper error or logging practices. Findings were validated through contextual analysis to reduce false positives and assess real-world exploitability. All identified issues were documented with clear descriptions, risk ratings, and remediation recommendations based on secure coding guidelines and applicable security standards.

With the expansion of blockchain technology and related integrations, source code review is a structured security assessment technique used to identify vulnerabilities, insecure coding practices, and logic flaws directly within an application’s source code. Unlike black-box testing, source code review provides full visibility into the application’s implementation, enabling the detection of security weaknesses that may not be observable during runtime testing. This approach is particularly effective for identifying issues related to authentication, authorization, data storage, cryptography, input validation, and error handling. The review is conducted in alignment with industry-recognized standards and best practices to ensure the application meets established security requirements.

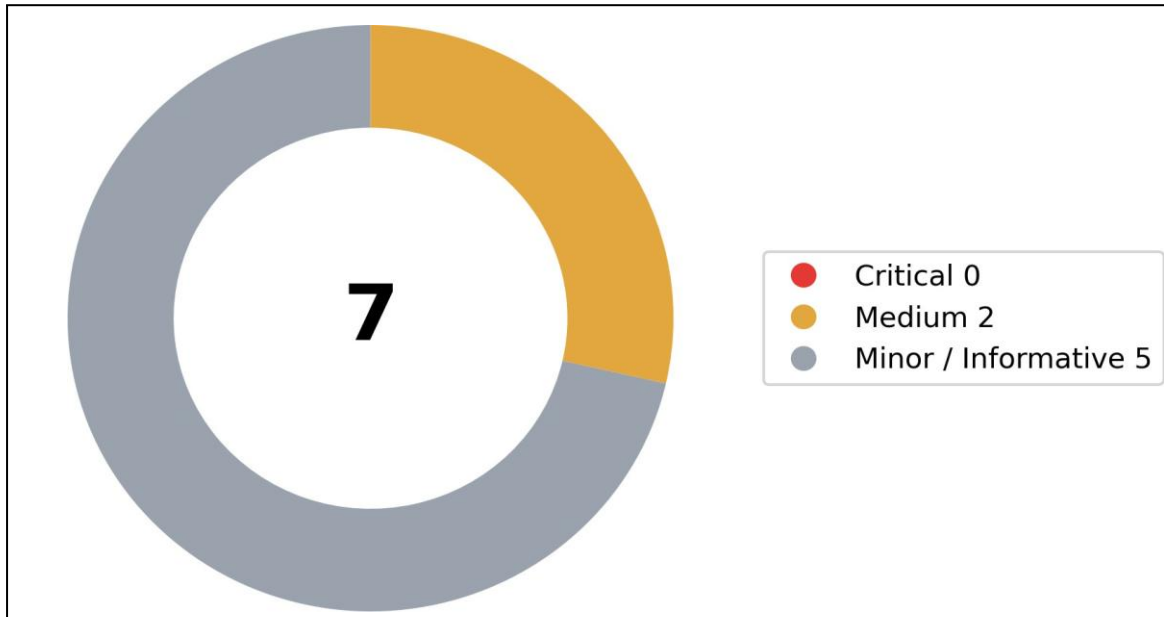
Source Code Review Scope

The scope of this assessment focused on identifying security vulnerabilities and weaknesses within the application’s source code and overall implementation, with the objective of providing actionable recommendations to improve its security posture. The review examined relevant portions of the codebase, with emphasis on security-critical components, including authentication and authorization logic, sensitive data handling, cryptographic implementations, input validation and output encoding, error handling, logging practices, and interaction with external services and dependencies.

Particular attention was given to common software security risks and insecure coding patterns referenced in industry standards such as OWASP Top 10, OWASP Mobile/Web Security Guidelines, and CWE, including hardcoded secrets, improper access control enforcement, insecure deserialization, injection vulnerabilities, security misconfigurations, and inadequate protection of sensitive information.

This report aims to provide a comprehensive understanding of the application's code-level security strengths and areas for improvement, enabling informed decisions to remediate identified issues, reduce attack surface, and enhance the overall security resilience of the application.

Findings Breakdown



Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	0	0	0
● Medium	0	0	2	0
● Minor / Informative	0	0	5	0

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	NDSWU	No Docker Secrets Were Used	Resolved
●	NAI	No Authentication Implemented	Resolved
●	BINPBD	Base Image is Not Pinned By Digest	Resolved
●	VOC	Vulnerable and Outdated Component	Resolved
●	SE	Swagger Enabled	Resolved
●	ACAOHWW	Access-Control-Allow-Origin Header with Wildcard.	Resolved
●	IME	Insecure Methods Enabled	Resolved

NDSWU - No Docker Secrets Were Used

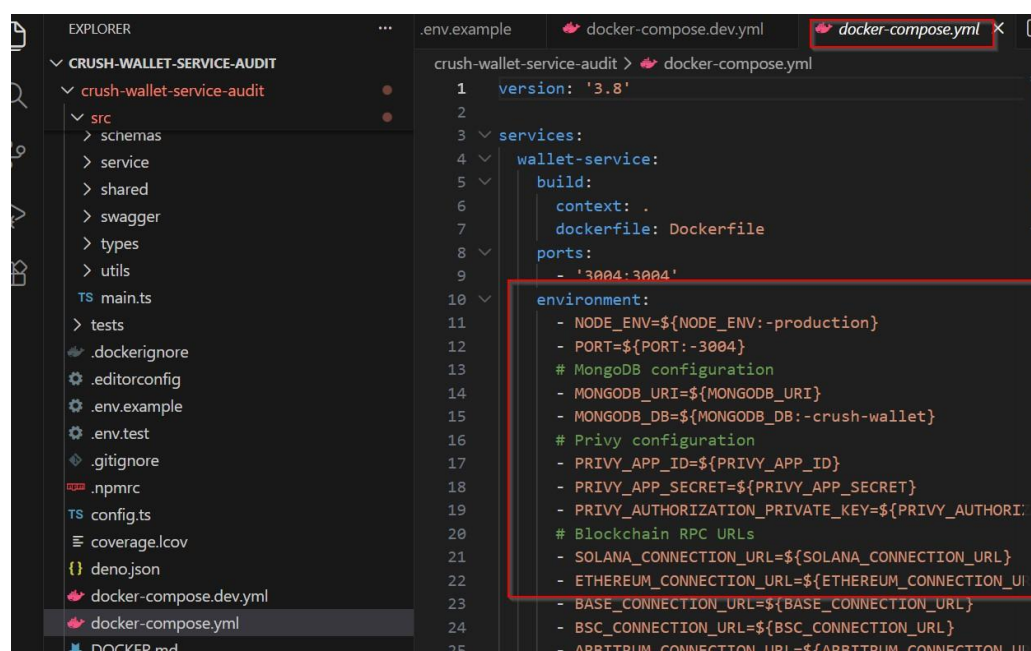
Criticality	Medium
Status	Resolved
CWE ID:	522
CVSS3 Score:	5.5

Description

The review identified that the application container configuration does not utilize Docker Secrets for managing sensitive information such as credentials, API keys, or tokens. Instead, sensitive data appears to be embedded directly within configuration files, environment variables, or container build instructions. This practice increases the exposure risk of secrets during image distribution, deployment, and runtime operations.

Storing secrets directly within container configurations or environment variables increases the likelihood of credential exposure through container image repositories, logs, backups, or compromised containers. Attackers gaining access to the container or image registry may retrieve sensitive credentials, potentially enabling unauthorized access to backend services, databases, or internal systems.

Proof of Concept: The below screenshots depict that no docker secrets used in the following docker config file.

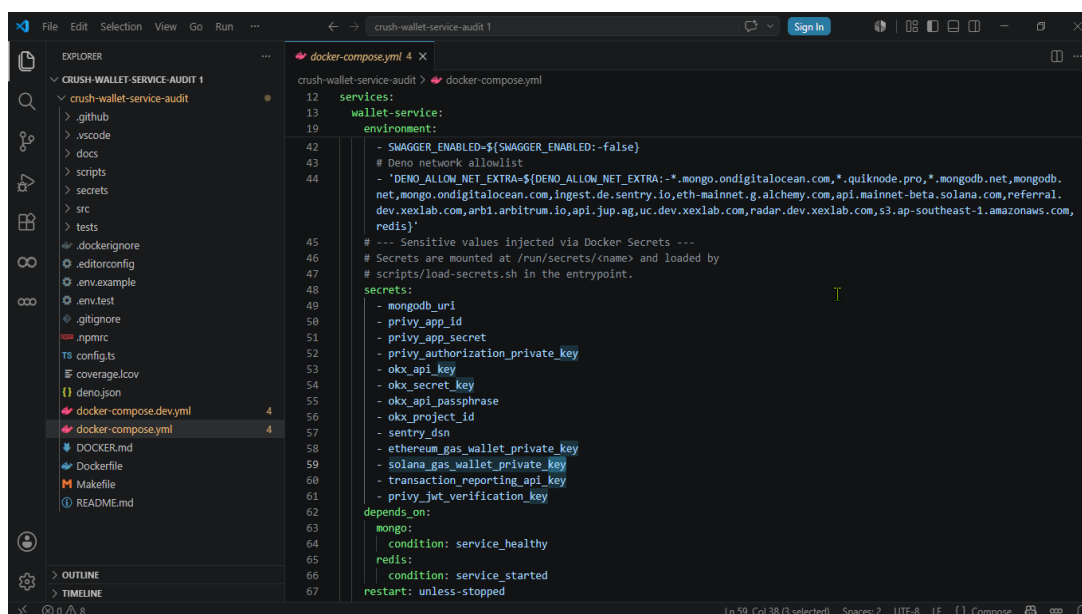


```
crush-wallet-service-audit > docker-compose.yml
1  version: '3.8'
2
3  services:
4    wallet-service:
5      build:
6        context: .
7        dockerfile: Dockerfile
8      ports:
9        - '3004:3004'
10     environment:
11       - NODE_ENV=${NODE_ENV:-production}
12       - PORT=${PORT:-3004}
13       # MongoDB configuration
14       - MONGODB_URI=${MONGODB_URI}
15       - MONGODB_DB=${MONGODB_DB:-crush-wallet}
16       # Privy configuration
17       - PRIVY_APP_ID=${PRIVY_APP_ID}
18       - PRIVY_APP_SECRET=${PRIVY_APP_SECRET}
19       - PRIVY_AUTHORIZATION_PRIVATE_KEY=${PRIVY_AUTHORI:
20       # Blockchain RPC URLs
21       - SOLANA_CONNECTION_URL=${SOLANA_CONNECTION_URL}
22       - ETHEREUM_CONNECTION_URL=${ETHEREUM_CONNECTION_UI
23       - BASE_CONNECTION_URL=${BASE_CONNECTION_URL}
24       - BSC_CONNECTION_URL=${BSC_CONNECTION_URL}
25       - ARBITRUM_CONNECTION_URL=${ARBITRUM_CONNECTION_UI
```

Recommendation

It is recommended to implement Docker Secrets or a secure secrets management solution such as HashiCorp Vault, Kubernetes Secrets, or cloud-based secret managers. Sensitive values should not be embedded in images or source code. Access to secrets should follow the principle of least privilege and be dynamically injected at runtime.

Revalidated Proof of Concept: In production, the wallet service uses Docker Secrets. The docker-compose.yml file loads all sensitive values, including the database URL, API keys, private keys, Sentry DSN, and other secrets, from Docker Secrets at runtime. As a result, sensitive information is not hardcoded in the compose file.



```
crush-wallet-service-audit > docker-compose.yml
12 services:
13   wallet-service:
14     environment:
15       - SWAGGER_ENABLED=${SWAGGER_ENABLED:-false}
16       - Deno network allowlist
17       - "DENO_ALLOW_NET_EXTRA=${DENO_ALLOW_NET_EXTRA:-*.mongo.ondigitalocean.com,*.quiknode.pro,*.mongodb.net,mongodb.
18         net,mongo.ondigitalocean.com,ingest.de.sentry.io,eth-mainnet.g.alchemy.com,api.mainnet-beta.solana.com,referral.
19         dev.xexlab.com,arbi.arbitrum.io,api.jup.ag,uc.dev.xexlab.com,radar.dev.xexlab.com,s3.ap-southeast-1.amazonaws.com,
20         redis}"
21       # --- Sensitive values injected via Docker Secrets ---
22       # Secrets are mounted at /run/secrets/<name> and loaded by
23       # scripts/load-secrets.sh in the entrypoint.
24     secrets:
25       - mongodb_uri
26       - privy_app_id
27       - privy_app_secret
28       - privy_authorization_private_key
29       - olx_api_key
30       - olx_secret_key
31       - olx_api_passphrase
32       - olx_project_id
33       - sentry_dsn
34       - ethereum_gas_wallet_private_key
35       - solana_gas_wallet_private_key
36       - transaction_reporting_api_key
37       - privy_jwt_verification_key
38     depends_on:
39       mongo:
40         condition: service_healthy
41       redis:
42         condition: service_started
43     restart: unless-stopped
```

NAI - No Authentication Implemented

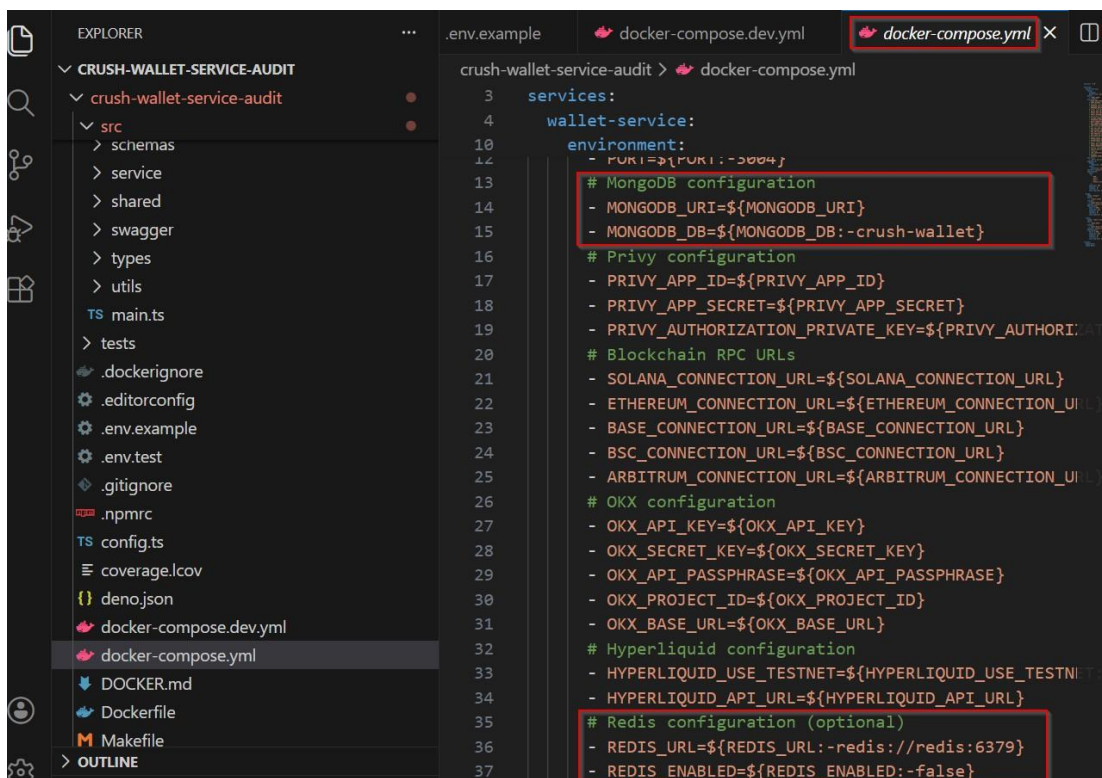
Criticality	Medium
Status	Resolved
CWE ID:	306
CVSS3 Score:	6.5

Description

The source code analysis revealed that certain application endpoints or functionalities are accessible without any authentication or identity validation mechanism. These endpoints allow users to access resources or perform operations without verifying their identity, indicating that authentication controls are either missing or improperly implemented.

The absence of authentication mechanisms can allow unauthorized users to access sensitive application features or data. Attackers may exploit such endpoints to perform unauthorized operations, extract confidential information, manipulate application behavior, or potentially escalate the attack surface leading to broader compromise of the application environment.

Proof of Concept: The below screenshot depicts that authentication is not implemented.

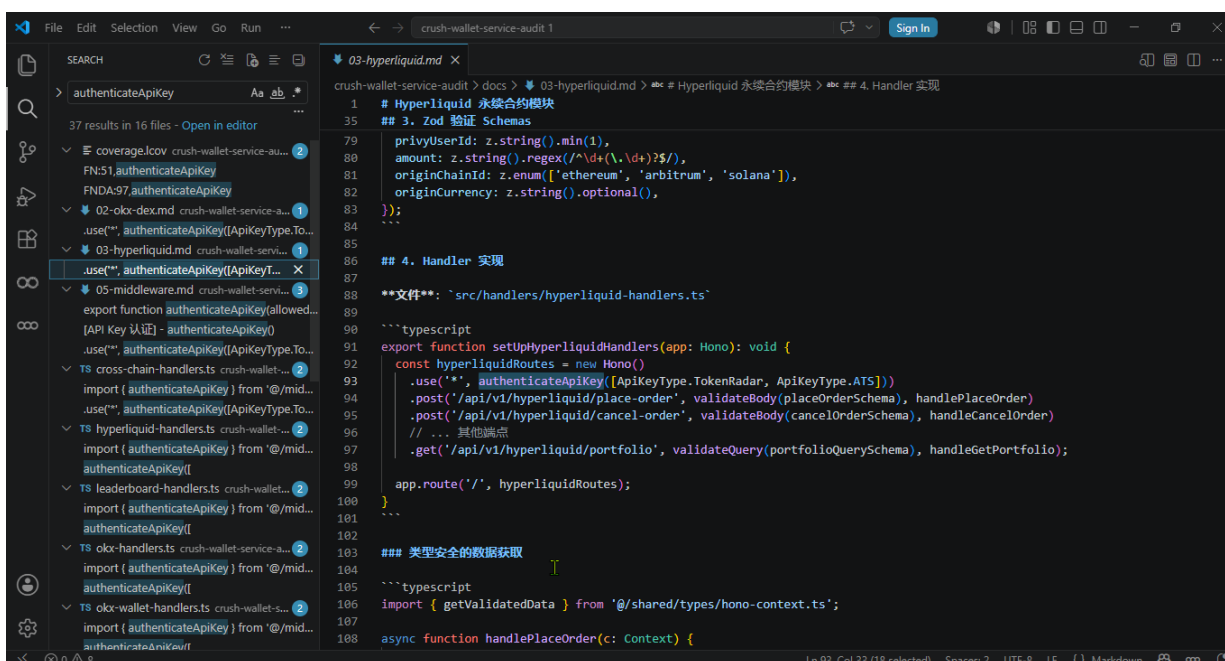


```
crush-wallet-service-audit > docker-compose.yml
3  services:
4    wallet-service:
5      environment:
6        - FURI=${FURI:-5004}
7        # MongoDB configuration
8        - MONGODB_URI=${MONGODB_URI}
9        - MONGODB_DB=${MONGODB_DB:-crush-wallet}
10       # Privy configuration
11       - PRIVY_APP_ID=${PRIVY_APP_ID}
12       - PRIVY_APP_SECRET=${PRIVY_APP_SECRET}
13       - PRIVY_AUTHORIZATION_PRIVATE_KEY=${PRIVY_AUTHORIZATION_PRIVATE_KEY}
14       # Blockchain RPC URLs
15       - SOLANA_CONNECTION_URL=${SOLANA_CONNECTION_URL}
16       - ETHEREUM_CONNECTION_URL=${ETHEREUM_CONNECTION_URL}
17       - BASE_CONNECTION_URL=${BASE_CONNECTION_URL}
18       - BSC_CONNECTION_URL=${BSC_CONNECTION_URL}
19       - ARBITRUM_CONNECTION_URL=${ARBITRUM_CONNECTION_URL}
20       # OKX configuration
21       - OKX_API_KEY=${OKX_API_KEY}
22       - OKX_SECRET_KEY=${OKX_SECRET_KEY}
23       - OKX_API_PASSPHRASE=${OKX_API_PASSPHRASE}
24       - OKX_PROJECT_ID=${OKX_PROJECT_ID}
25       - OKX_BASE_URL=${OKX_BASE_URL}
26       # Hyperliquid configuration
27       - HYPERLIQUID_USE_TESTNET=${HYPERLIQUID_USE_TESTNET}
28       - HYPERLIQUID_API_URL=${HYPERLIQUID_API_URL}
29       # Redis configuration (optional)
30       - REDIS_URL=${REDIS_URL:-redis://redis:6379}
31       - REDIS_ENABLED=${REDIS_ENABLED:-false}
```

Recommendation

It is recommended to implement robust authentication mechanisms such as token-based authentication, OAuth2, or session-based authentication depending on the application architecture. Access to all critical endpoints should require identity verification. Additionally, enforce role-based or attribute-based access control to ensure only authorized users can access sensitive functionality.

Revalidated Proof of Concept: The below screenshot depicts authentication is implemented through the `authenticateApiKey` middleware, which is applied to the API routes, ensuring that requests are authenticated before accessing protected endpoints.



The screenshot displays a code editor with a search bar on the left and a code editor on the right. The search bar shows results for `authenticateApiKey` across 16 files. The code editor shows the implementation of the `authenticateApiKey` middleware and its application to Hyperliquid API routes.

```
1 # Hyperliquid 永续合约模块
35 ## 3. Zod 验证 Schemas
79 privityUserId: z.string().min(1),
80 amount: z.string().regex(/^(\d+(\.\d+)?)$/),
81 originChainId: z.enum(['ethereum', 'arbitrum', 'solana']),
82 originCurrency: z.string().optional(),
83 });
84 ...
85
86 ## 4. Handler 实现
87
88 **文件**: `src/handlers/hyperliquid-handlers.ts`
89
90 ```typescript
91 export function setUpHyperliquidHandlers(app: Hono): void {
92   const hyperliquidRoutes = new Hono()
93   .use('*', authenticateApiKey([ApiKeyType.TokenRadar, ApiKeyType.ATS]))
94   .post('/api/v1/hyperliquid/place-order', validateBody(placeOrderSchema), handlePlaceOrder)
95   .post('/api/v1/hyperliquid/cancel-order', validateBody(cancelOrderSchema), handleCancelOrder)
96   // ... 其他端点
97   .get('/api/v1/hyperliquid/portfolio', validateQuery(portfolioQuerySchema), handleGetPortfolio);
98
99   app.route('/', hyperliquidRoutes);
100 }
101 ...
102
103 ## 类型安全的数据获取
104
105 ```typescript
106 import { getValidatedData } from '@shared/types/hono-context.ts';
107
108 async function handlePlaceOrder(c: Context) {
```

BINPBD - Base Image is Not Pinned By Digest

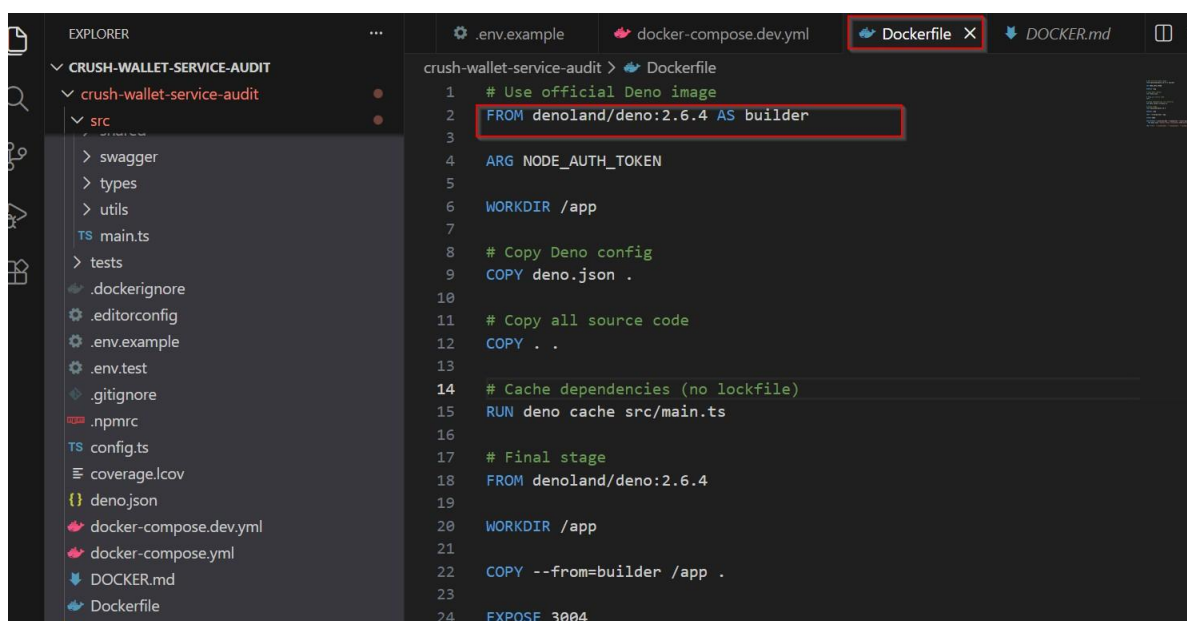
Criticality	Minor
Status	Resolved
CWE ID:	1104
CVSS3 Score:	3.7

Description

The Docker configuration uses a base image referenced only by a tag (e.g., latest or version tag) instead of a specific immutable image digest. Tags can change over time, causing builds to pull different image versions during each build process, which may introduce unintended or unverified updates into the container environment.

Using mutable image tags may result in inconsistent builds and potential introduction of malicious or vulnerable image versions. If a base image repository is compromised or updated unexpectedly, the application could unknowingly inherit insecure components, affecting the stability, integrity, and security of the deployed container.

Proof of Concept: The below screenshot depicts that Base Image is Not Pinned By Digest.



```

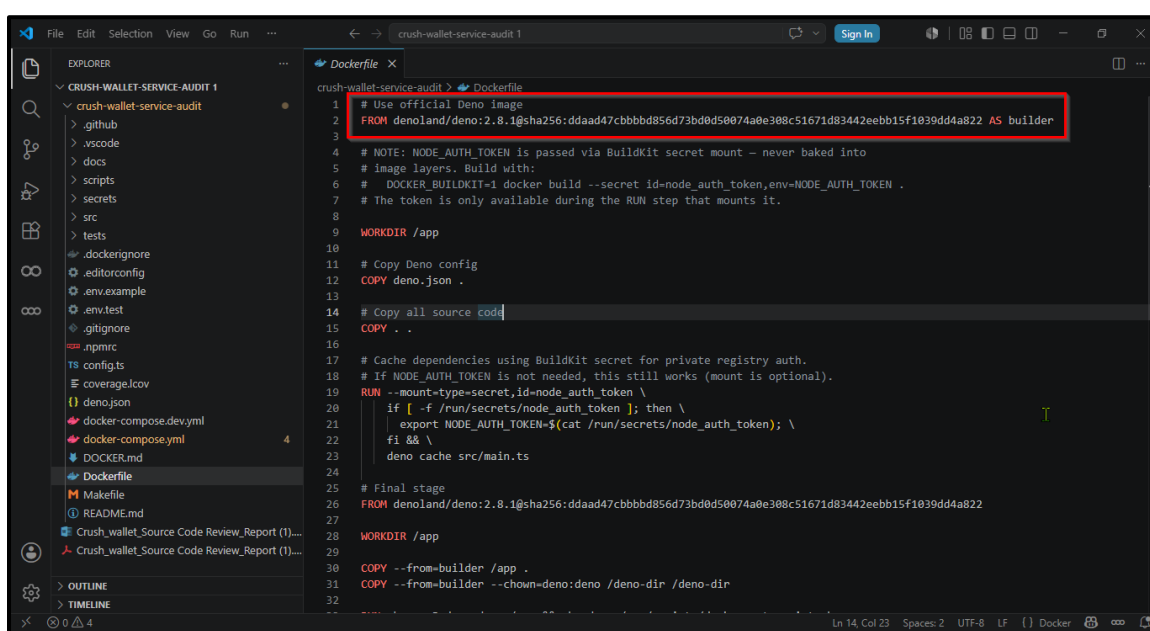
1  # Use official Deno image
2  FROM denoland/deno:2.6.4 AS builder
3
4  ARG NODE_AUTH_TOKEN
5
6  WORKDIR /app
7
8  # Copy Deno config
9  COPY deno.json .
10
11 # Copy all source code
12 COPY . .
13
14 # Cache dependencies (no lockfile)
15 RUN deno cache src/main.ts
16
17 # Final stage
18 FROM denoland/deno:2.6.4
19
20 WORKDIR /app
21
22 COPY --from=builder /app .
23
24 EXPOSE 3004

```

Recommendation

It is recommended to pin Docker base images using a specific image digest (SHA256). This ensures consistent and reproducible builds by always referencing the exact same base image. Additionally, implement automated container image vulnerability scanning and regularly update base images through controlled and verified processes.

Revalidated Proof of Concept: The below screenshot depicts that Base Image is Pinned By Digest.



The screenshot shows a code editor with a Dockerfile open. The file is named 'Dockerfile' and is located in the 'crush-wallet-service-audit' directory. The Dockerfile content is as follows:

```
1 # Use official Deno image
2 FROM denoland/deno:2.8.1@sha256:ddaad47cbbbd856d73bd0d50074a0e308c51671d83442eebb15f1039dd4a822 AS builder
3
4 # NOTE: NODE_AUTH_TOKEN is passed via BuildKit secret mount - never baked into
5 # image layers. Build with:
6 # DOCKER_BUILDKIT=1 docker build --secret id=node_auth_token,env=NODE_AUTH_TOKEN .
7 # The token is only available during the RUN step that mounts it.
8
9 WORKDIR /app
10
11 # Copy Deno config
12 COPY deno.json .
13
14 # Copy all source code
15 COPY . .
16
17 # Cache dependencies using BuildKit secret for private registry auth.
18 # If NODE_AUTH_TOKEN is not needed, this still works (mount is optional).
19 RUN --mount=type=secret,id=node_auth_token \
20     if [ -f /run/secrets/node_auth_token ]; then \
21         export NODE_AUTH_TOKEN=$(cat /run/secrets/node_auth_token); \
22         fi && \
23     deno cache src/main.ts
24
25 # Final stage
26 FROM denoland/deno:2.8.1@sha256:ddaad47cbbbd856d73bd0d50074a0e308c51671d83442eebb15f1039dd4a822
27
28 WORKDIR /app
29
30 COPY --from=builder /app .
31 COPY --from=builder --chown=deno:deno /deno-dir /deno-dir
```

The line `FROM denoland/deno:2.8.1@sha256:ddaad47cbbbd856d73bd0d50074a0e308c51671d83442eebb15f1039dd4a822 AS builder` is highlighted with a red box, indicating that the base image is pinned by its SHA256 digest.

VOC - Vulnerable and Outdated Component

Criticality	Minor
Status	Resolved
CWE ID:	1104
CVSS3 Score:	4.0

Description

The source code review identified that the application is using an outdated version of the Deno JavaScript runtime, specifically version 2.6.4+, which has reached end of security support. Outdated runtime components may contain publicly disclosed vulnerabilities and security weaknesses that are addressed in later versions, potentially exposing the application environment to known security risks.

Using an unsupported or outdated runtime such as Deno 2.6.4 may expose the application to known vulnerabilities that could be exploited by attackers. These vulnerabilities may allow attackers to perform unauthorized actions, compromise application integrity, or affect the availability of services if exploited through runtime weaknesses or dependent components.

Proof of Concept: The below screenshot depicts that outdated deno 2.6.4 version is used which does not have any security support available.


```
1 # Crush Wallet Service
2
3 A scalable wallet and trading service built with Deno, providing wallet operations and trading
  using Privy for server-side signing.
4
5 ## Features
6
7 - **Wallet Operations**: Get wallet addresses, token balances, and wallet activities
8 - **Trading**: OKX DEX swaps and Hyperliquid perpetual futures trading
9 - **Privy Integration**: Server-side signing using Privy session signers
10 - **Multi-chain Support**: Solana, Ethereum, Base, BSC, Arbitrum, Hyperliquid
11 - **API Key Authentication**: Secure API access with key-based authentication
12 - **Swagger Documentation**: Interactive API documentation
13
14 ## Tech Stack
15
16 - **Runtime**: Deno 2.6.4+
17 - **Framework**: Hono
18 - **Database**: MongoDB
19 - **Cache**: Redis (optional)
20 - **Validation**: Zod
21 - **Blockchain**: Ethers.js, @solana/web3.js
22 - **Wallet**: Privy
23
24 ## Quick Start
25
26 ### Prerequisites
27
28 - Deno 2.6.4 or higher
29 - MongoDB instance (local or cloud)
30 - Redis (optional, for caching)
31 - Privy account with authorization keys
```

Release	Released	Security Support	Latest
2.7	5 days ago (25 Feb 2026)	Yes	2.7.1 (25 Feb 2026)
2.6	2 months and 3 weeks ago (10 Dec 2025)	Ended 5 days ago (25 Feb 2026)	2.6.10 (17 Feb 2026)
2.5 (LTS)	5 months and 3 weeks ago (10 Sep 2025)	Ends in 2 months (30 Apr 2026)	2.5.7 (27 Jan 2026)
2.4	8 months ago (01 Jul 2025)	Ended 5 months and 3 weeks ago (10 Sep 2025)	2.4.5 (21 Aug 2025)

Show more unmaintained releases

Recommendation

It is recommended to upgrade the runtime to a supported and actively maintained version of Deno, preferably the latest stable release. Keeping runtime environments updated ensures that known vulnerabilities are patched. Additionally, implement a dependency

monitoring process to regularly track security advisories and apply updates in a timely manner.

Revalidated Proof of Concept: The below screenshot depicts that deno 2.8.3 version is used which has security support available.

```
1 # Crush Wallet Service
2
3 ## Features
4
5 - **Wallet Operations**: Get wallet addresses, token balances, and wallet activities
6 - **Trading**: OKX DEX swaps and Hyperliquid perpetual futures trading
7 - **Privy Integration**: Server-side signing using Privy session signers
8 - **Multi-chain Support**: Solana, Ethereum, Base, BSC, Arbitrum, Hyperliquid
9 - **API Key Authentication**: Secure API access with key-based authentication
10 - **Swagger Documentation**: Interactive API documentation
11
12 ## Tech Stack
13
14 - **Runtime**: Deno 2.8.14
15 - Framework: None
16 - **Database**: MongoDB
17 - **Cache**: Redis (optional)
18 - **Validation**: Zod
19 - **Blockchain**: Ethers.js, @solana/web3.js
20 - **Wallet**: Privy
21
22 ## Quick Start
23
24 ## Prerequisites
25
26 - Deno 2.8.1 or higher
27 - MongoDB instance (local or cloud)
28 - Redis (optional, for caching)
29 - Privy account with authorization keys
30
31 ## Installation
32
33 1. **Clone the repository**:
```

Deno (FRAMEWORK | JAVASCRIPT-RUNTIME)

Last updated on 07 June 2026

Deno is a JavaScript, TypeScript, and WebAssembly runtime with secure defaults and a great developer experience. It's built on V8, Rust, and Tokio.

Timeline chart showing versions 2.5 (LTS), 2.6, 2.7, and 2.8. Version 2.8 is highlighted with a red box.

Release	Released	Security Support	Latest
2.8	2 weeks and 4 days ago (22 May 2026)	Yes	2.8.2 (03 Jun 2026)
2.7	3 months and 2 weeks ago (25 Feb 2026)	Ended 2 weeks and 4 days ago (22 May 2026)	2.7.14 (28 Apr 2026)

Show more unmaintained releases

SE - Swagger Enabled

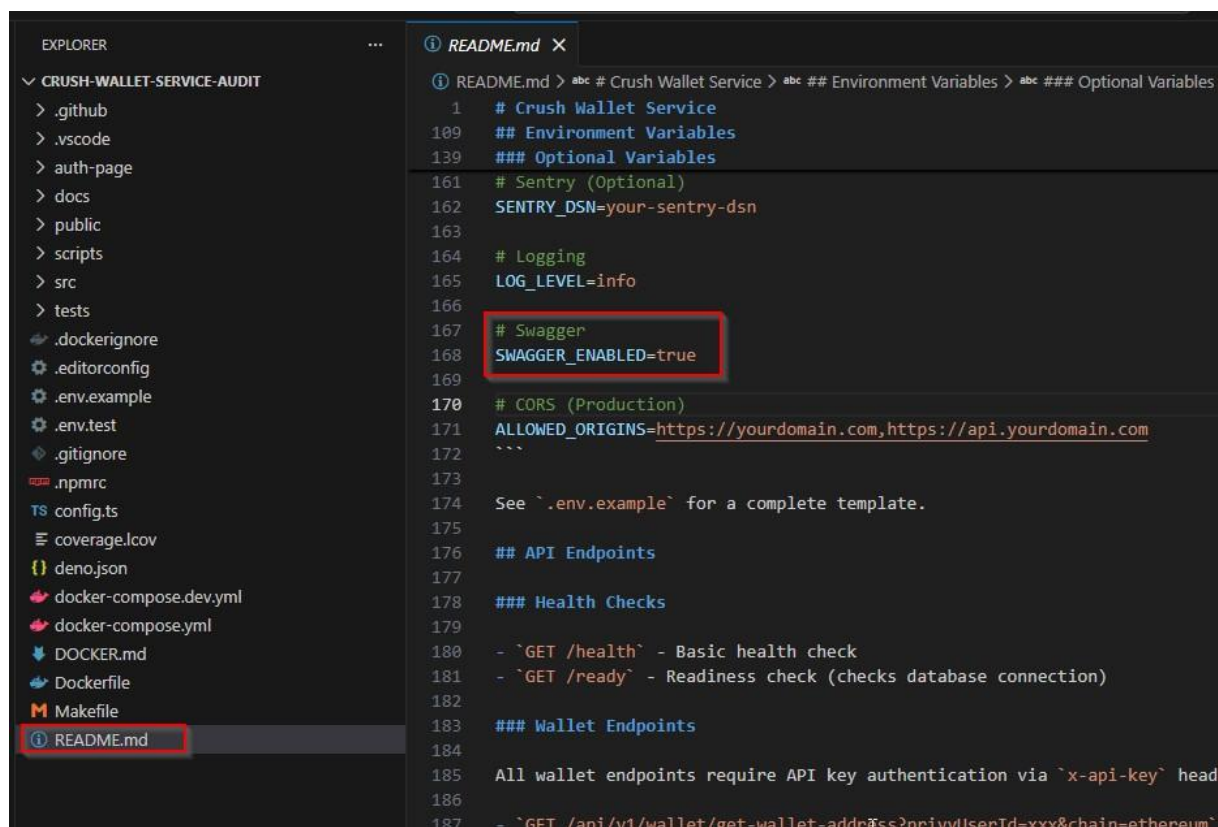
Criticality	Minor
Status	Resolved
CWE ID:	200
CVSS3 Score:	3.7

Description

The analysis identified that Swagger API documentation is enabled within the application environment. Swagger provides interactive API documentation that exposes endpoint details, request structures, parameters, and response formats, which may reveal internal API functionality and implementation details.

If exposed in production environments, Swagger documentation can assist attackers in enumerating available API endpoints and understanding application functionality. This information may facilitate targeted attacks, such as API abuse, injection attempts, or authentication bypass attempts, especially if security controls are not properly implemented.

Proof of Concept: The below screenshot depicts that SWAGGER_ENABLED is set to True.



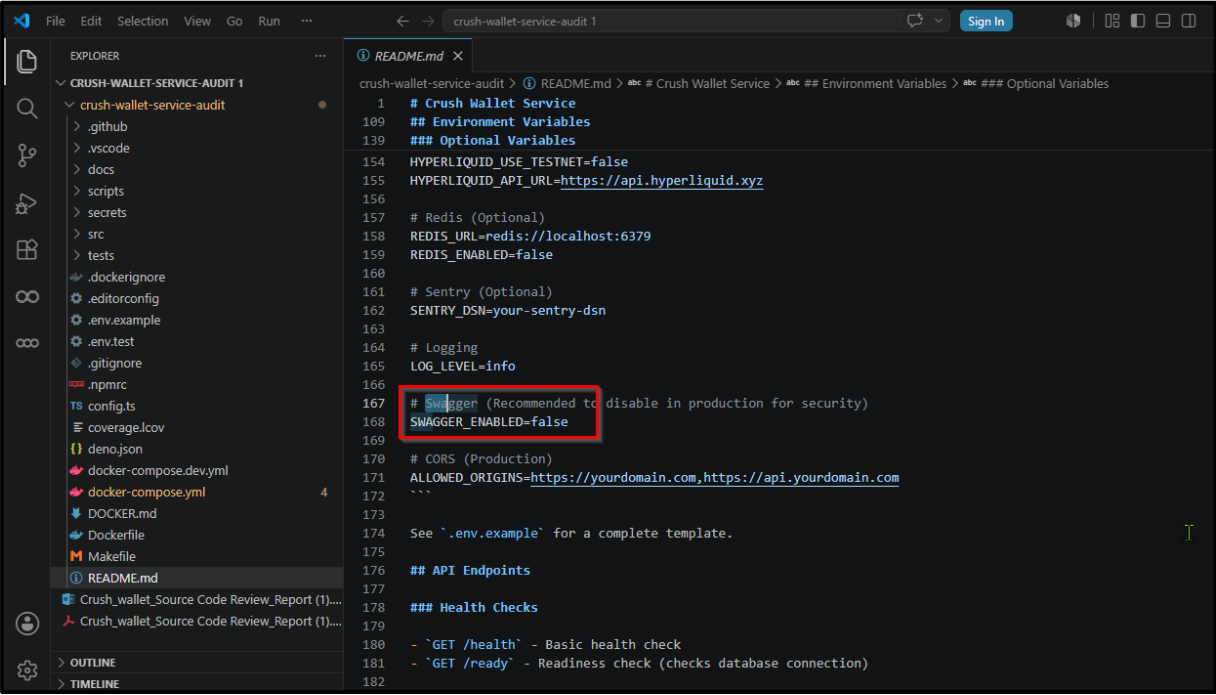
```

1 # Crush Wallet Service
109 ## Environment Variables
139 ### Optional Variables
161 # Sentry (Optional)
162 SENTRY_DSN=your-sentry-dsn
163
164 # Logging
165 LOG_LEVEL=info
166
167 # Swagger
168 SWAGGER_ENABLED=true
169
170 # CORS (Production)
171 ALLOWED_ORIGINS=https://yourdomain.com,https://api.yourdomain.com
172 ```
173
174 See `.env.example` for a complete template.
175
176 ## API Endpoints
177
178 ### Health Checks
179
180 - `GET /health` - Basic health check
181 - `GET /ready` - Readiness check (checks database connection)
182
183 ### Wallet Endpoints
184
185 All wallet endpoints require API key authentication via `x-api-key` head
186
187 - `GET /api/v1/wallet/get-wallet-address?privyUserId=xxx&chain=ethereum`
  
```

Recommendation

Swagger documentation should be disabled in production environments or restricted using authentication and network access controls. If documentation is required for operational purposes, it should only be accessible to authorized users within secure internal networks or protected development environments.

Revalidated Proof of Concept: The below screenshot depicts that SWAGGER_ENABLED is set to False.



```
crush-wallet-service-audit > ① README.md X
crush-wallet-service-audit > ① README.md > abc # Crush Wallet Service > abc ## Environment Variables > abc ### Optional Variables
1 # Crush Wallet Service
109 ## Environment Variables
139 ### Optional Variables
154 HYPERLIQUID_USE_TESTNET=false
155 HYPERLIQUID_API_URL=https://api.hyperliquid.xyz
156
157 # Redis (Optional)
158 REDIS_URL=redis://localhost:6379
159 REDIS_ENABLED=false
160
161 # Sentry (Optional)
162 SENTRY_DSN=your-sentry-dsn
163
164 # Logging
165 LOG_LEVEL=info
166
167 # Swagger (Recommended to disable in production for security)
168 SWAGGER_ENABLED=false
169
170 # CORS (Production)
171 ALLOWED_ORIGINS=https://yourdomain.com,https://api.yourdomain.com
172
173
174 See `.env.example` for a complete template.
175
176 ## API Endpoints
177
178 ### Health Checks
179
180 - `GET /health` - Basic health check
181 - `GET /ready` - Readiness check (checks database connection)
182
```

ACAOHWW - Access-Control-Allow-Origin Header with Wildcard.

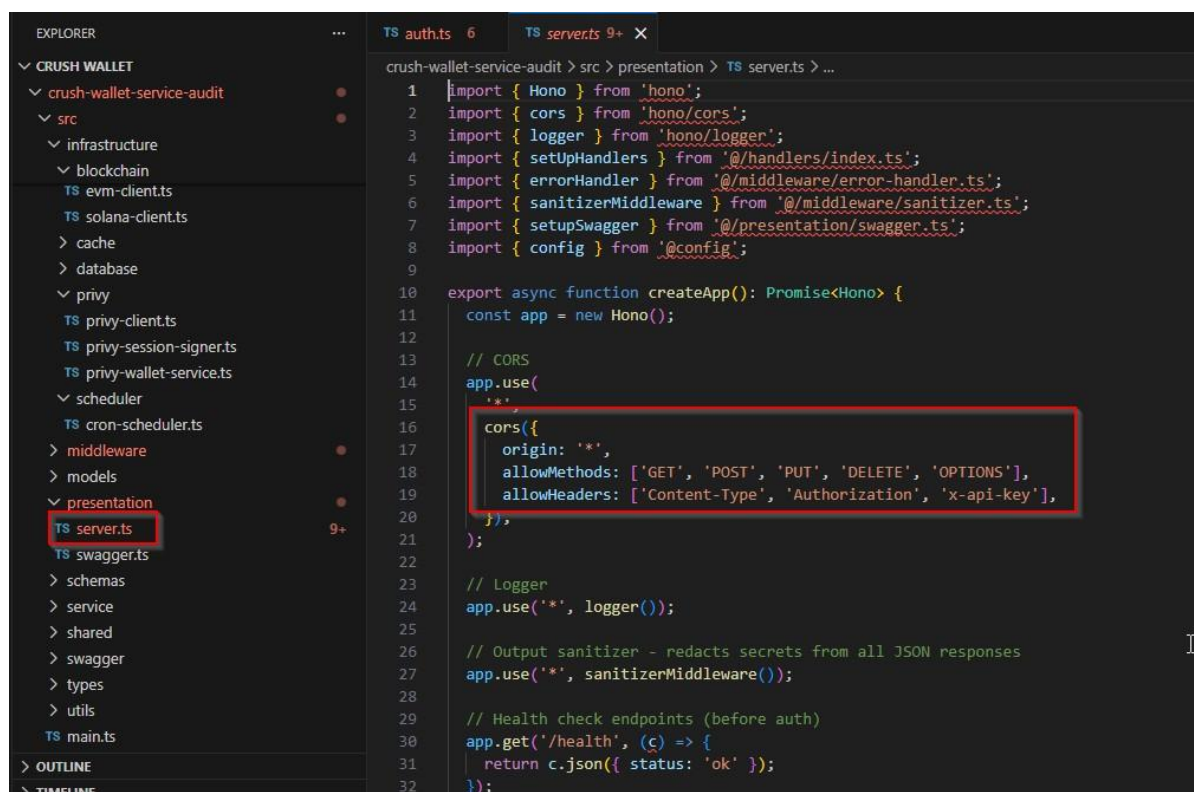
Criticality	Minor
Status	Resolved
CWE ID:	942
CVSS3 Score:	4.3

Description

The application response headers include the Access-Control-Allow-Origin header configured with a wildcard ("*"), allowing any external domain to make cross-origin requests to the application resources. This configuration permits unrestricted cross-domain interactions without validating trusted origins.

Permissive CORS configurations may allow malicious websites to interact with application resources through a victim's browser. If combined with authenticated sessions or sensitive APIs, attackers may exploit this configuration to perform unauthorized actions or retrieve sensitive data via cross-origin requests.

Proof of Concept: The below screenshot depicts that the Origin header is set to wildcard.



```
EXPLORER
  CRUSH WALLET
    crush-wallet-service-audit
      src
        infrastructure
          blockchain
            TS evm-client.ts
            TS solana-client.ts
          > cache
          > database
          > privy
            TS privy-client.ts
            TS privy-session-signer.ts
            TS privy-wallet-service.ts
          > scheduler
            TS cron-scheduler.ts
          > middleware
          > models
          > presentation
            TS server.ts
            TS swagger.ts
          > schemas
          > service
          > shared
          > swagger
          > types
          > utils
        TS main.ts
      > OUTLINE
      > TIMELINE

TS auth.ts 6
TS server.ts 9+ X

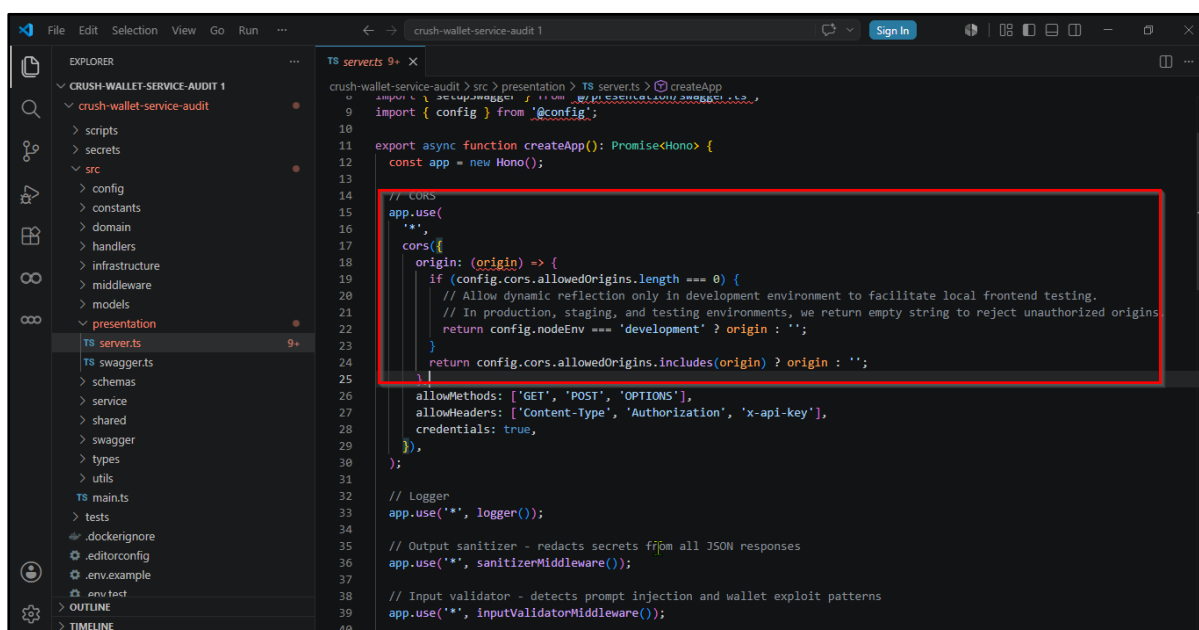
crush-wallet-service-audit > src > presentation > TS server.ts > ...

1  import { Hono } from 'hono';
2  import { cors } from 'hono/cors';
3  import { logger } from 'hono/logger';
4  import { setUpHandlers } from '@handlers/index.ts';
5  import { errorHandler } from '@middleware/error-handler.ts';
6  import { sanitizerMiddleware } from '@middleware/sanitizer.ts';
7  import { setupSwagger } from '@presentation/swagger.ts';
8  import { config } from '@config';
9
10 export async function createApp(): Promise<Hono> {
11   const app = new Hono();
12
13   // CORS
14   app.use(
15     '*',
16     cors({
17       origin: '*',
18       allowMethods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
19       allowHeaders: ['Content-Type', 'Authorization', 'x-api-key'],
20     }),
21   );
22
23   // Logger
24   app.use('*', logger());
25
26   // Output sanitizer - redacts secrets from all JSON responses
27   app.use('*', sanitizerMiddleware());
28
29   // Health check endpoints (before auth)
30   app.get('/health', (c) => {
31     return c.json({ status: 'ok' });
32   });
```

Recommendation

It is recommended to restrict the Access-Control-Allow-Origin header to specific trusted domains instead of using a wildcard. Only legitimate client domains that require access should be explicitly whitelisted. Additionally, implement proper authentication and validation controls to ensure secure cross-origin communication.

Revalidated Proof of Concept: The below screenshot depicts that the implementation has been updated to enforce an explicit origin allowlist via `config.cors.allowedOrigins`. Requests from unapproved origins are no longer allowed.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure for 'crush-wallet-service-audit'. The code editor shows the file 'server.ts' with the following code:

```
crush-wallet-service-audit > src > presentation > TS server.ts > createApp
import { config } from '@config';

export async function createApp(): Promise<Hono> {
  const app = new Hono();

  // CORS
  app.use(
    '*',
    cors({
      origin: (origin) => {
        if (config.cors.allowedOrigins.length === 0) {
          // Allow dynamic reflection only in development environment to facilitate local frontend testing.
          // In production, staging, and testing environments, we return empty string to reject unauthorized origins
          return config.nodeEnv === 'development' ? origin : '';
        }
        return config.cors.allowedOrigins.includes(origin) ? origin : '';
      }
    })
  );

  allowMethods: ['GET', 'POST', 'OPTIONS'],
  allowHeaders: ['Content-Type', 'Authorization', 'x-api-key'],
  credentials: true,
});

// Logger
app.use('*', logger());

// Output sanitizer - redacts secrets from all JSON responses
app.use('*', sanitizerMiddleware());

// Input validator - detects prompt injection and wallet exploit patterns
app.use('*', inputValidatorMiddleware());
```

IME - Insecure Methods Enabled

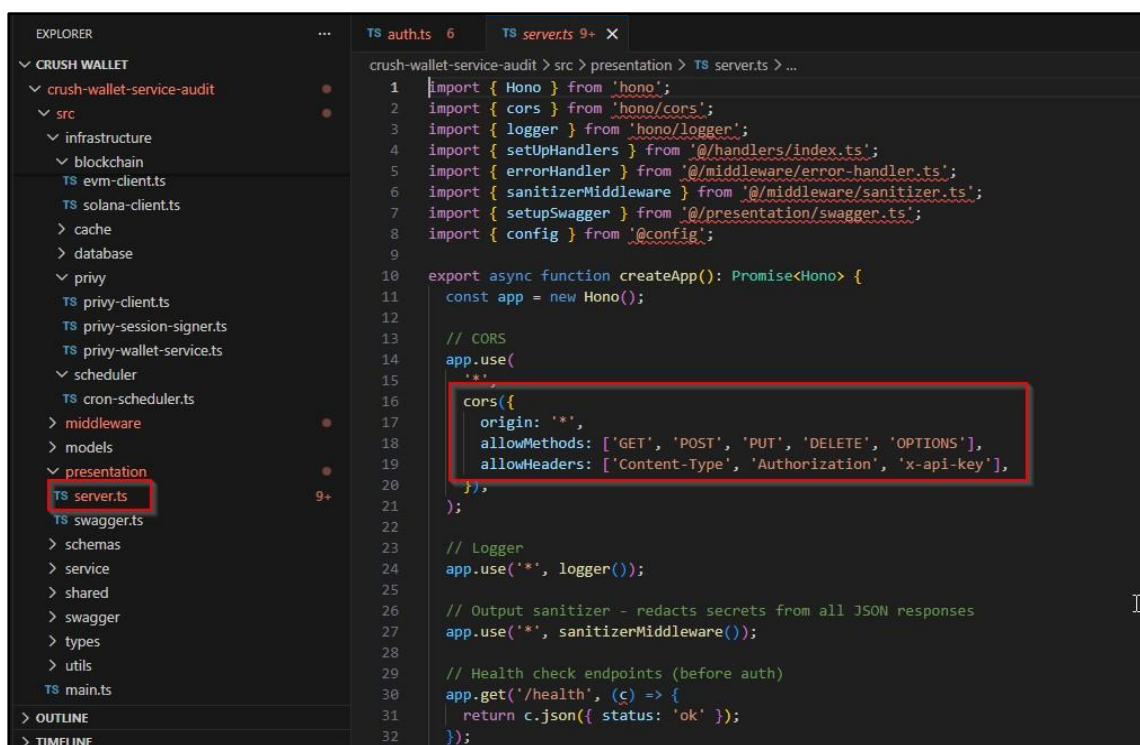
Criticality	Minor
Status	Resolved
CWE ID:	284
CVSS3 Score:	3.7

Description

The source code review identified that the application server allows additional HTTP methods such as PUT, DELETE, and OPTIONS. These methods are typically used for modifying or interacting with server resources and may not always be required for normal application functionality. Enabling unnecessary HTTP methods can increase the exposed attack surface of the application.

If these HTTP methods are not properly controlled or validated, attackers may attempt to exploit them to modify or delete application resources or perform unintended operations. Improper handling of PUT or DELETE requests may lead to unauthorized data manipulation, while the OPTIONS method can assist attackers in identifying supported methods during reconnaissance.

Proof of Concept: The below screenshot depicts that PUT, DELETE and OPTIONS methods are enabled.



```

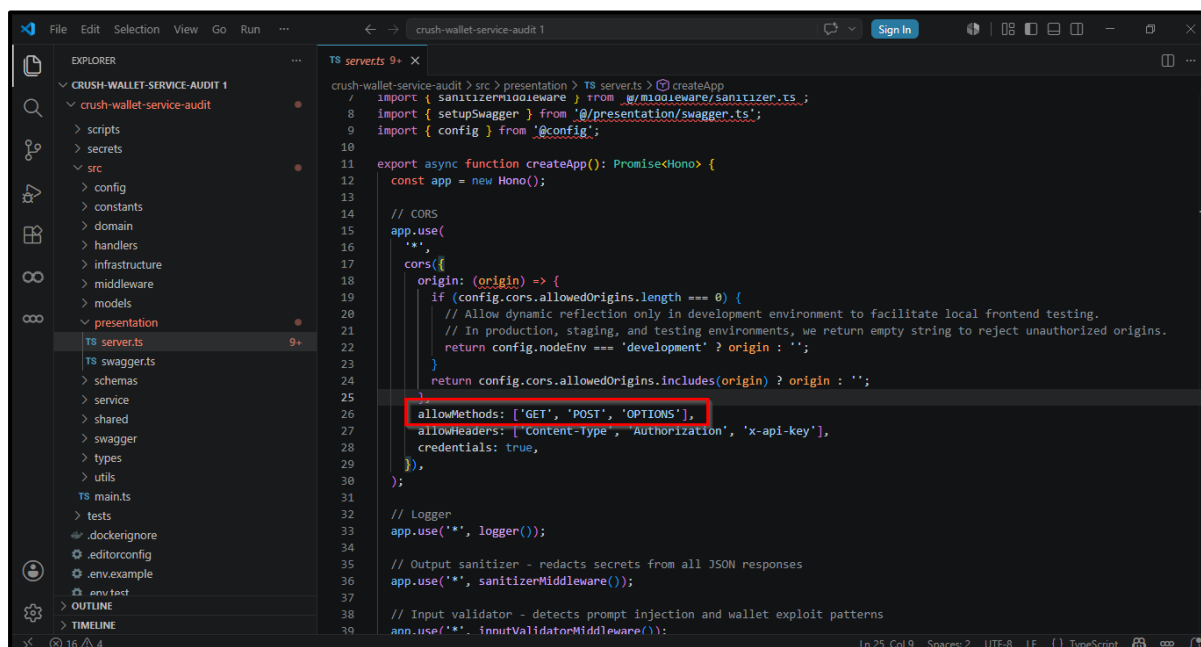
1  import { Hono } from 'hono';
2  import { cors } from 'hono/cors';
3  import { logger } from 'hono/logger';
4  import { setUpHandlers } from '@handlers/index.ts';
5  import { errorHandler } from '@middleware/error-handler.ts';
6  import { sanitizerMiddleware } from '@middleware/sanitizer.ts';
7  import { setupSwagger } from '@presentation/swagger.ts';
8  import { config } from '@config';
9
10 export async function createApp(): Promise<Hono> {
11   const app = new Hono();
12
13   // CORS
14   app.use(
15     '**',
16     cors({
17       origin: '**',
18       allowMethods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
19       allowHeaders: ['Content-Type', 'Authorization', 'x-api-key'],
20     }),
21   );
22
23   // Logger
24   app.use('**', logger());
25
26   // Output sanitizer - redacts secrets from all JSON responses
27   app.use('**', sanitizerMiddleware());
28
29   // Health check endpoints (before auth)
30   app.get('/health', (c) => {
31     return c.json({ status: 'ok' });
32   });

```

Recommendation

It is recommended to restrict HTTP methods to only those strictly required for application functionality. If PUT, DELETE, or OPTIONS methods are not necessary, they should be disabled at the web server or application configuration level. Additionally, implement proper authentication and authorization controls for any endpoints that legitimately require these methods.

Revalidated Proof of Concept: The below screenshot depicts that GET, POST and OPTIONS methods are enabled.



```
crush-wallet-service-audit > src > presentation > TS servers > createApp
/
import { sanitizerMiddleware } from '@middleware/sanitizer.ts';
8
import { setupSwagger } from '@presentation/swagger.ts';
9
import { config } from '@config';
10
11
export async function createApp(): Promise<Hono> {
12
  const app = new Hono();
13
  // CORS
14
  app.use(
15
    '*',
16
    cors({
17
      origin: (origin) => {
18
        if (config.cors.allowedOrigins.length === 0) {
19
          // Allow dynamic reflection only in development environment to facilitate local frontend testing.
20
          // In production, staging, and testing environments, we return empty string to reject unauthorized origins.
21
          return config.nodeEnv === 'development' ? origin : '';
22
        }
23
        return config.cors.allowedOrigins.includes(origin) ? origin : '';
24
      },
25
      allowMethods: ['GET', 'POST', 'OPTIONS'],
26
      allowHeaders: ['Content-type', 'Authorization', 'x-api-key'],
27
      credentials: true,
28
    }),
29
  );
30
  // Logger
31
  app.use('*', logger());
32
  // Output sanitizer - redacts secrets from all JSON responses
33
  app.use('*', sanitizerMiddleware());
34
  // Input validator - detects prompt injection and wallet exploit patterns
35
  app.use('*', inputValidatorMiddleware());
36
}
```


Summary

The source code review identified several security weaknesses related to dependency management, configuration practices, authentication controls, and API exposure. The application was found to use an outdated runtime component, specifically Deno version 2.6.4, which has reached the end of security support and may contain known vulnerabilities. Additionally, secure secret management mechanisms such as Docker Secrets were not implemented, increasing the risk of credential exposure. Certain application functionalities were observed without proper authentication controls, potentially allowing unauthorized access. The configuration also permits permissive Cross-Origin Resource Sharing through a wildcard origin, enables additional HTTP methods including PUT, DELETE, and OPTIONS, and exposes API documentation via Swagger in the environment. Furthermore, container configuration issues such as base images not pinned by digest were observed, which may lead to inconsistent or unverified builds. Collectively, these weaknesses may increase the attack surface and should be addressed to improve the overall security posture of the application and align with secure coding and deployment best practices.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io